Unity Workshop

Peter Soava

January 2026

SFU Game Dev's workshop for beginners to get started with Unity!

1 Introduction

Welcome to the Unity Workshop! In this lab, you will learn the essentials of using Unity, including placing objects, working with prefabs, adding a player, simple scripts, and basic gameplay mechanics. You will frequently interact with the **Scene View**, **Game View**, **Hierarchy**, **Inspector**, **Project / Assets Window**, and **Console** throughout the workshop.

Figure 1: Final game after we're done!

2 Unity Interface Overview

Before starting the lab, we'll go over the Unity interface.

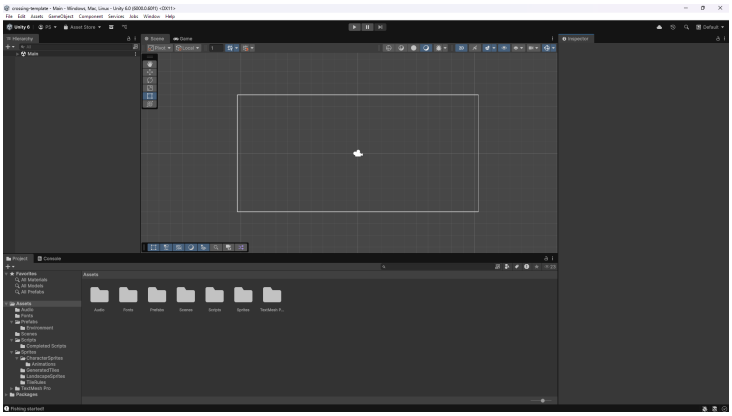
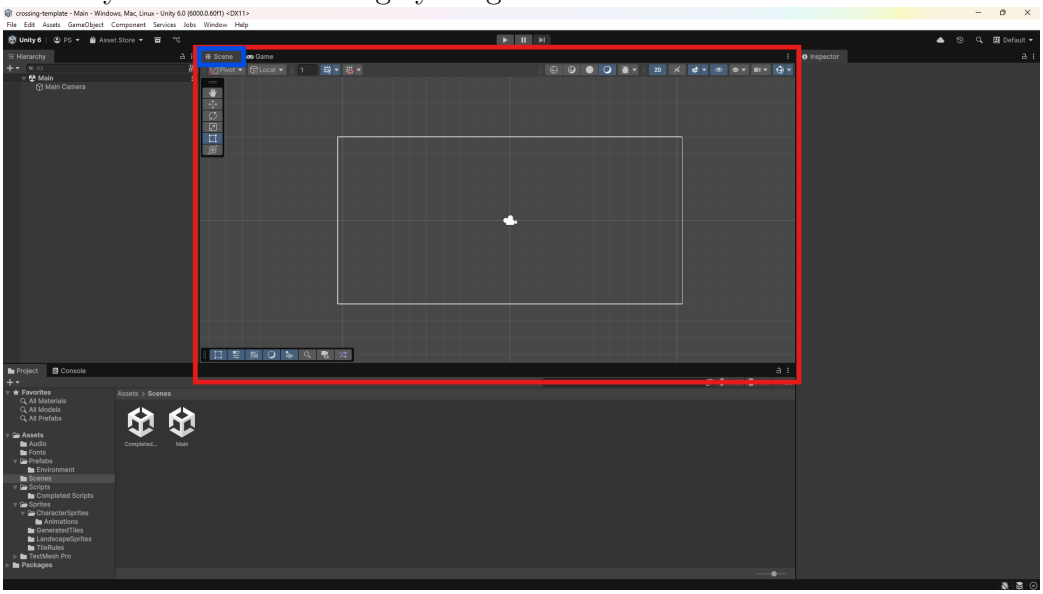


Figure 2: Unity Editor UI

2.1 Main Editor Windows

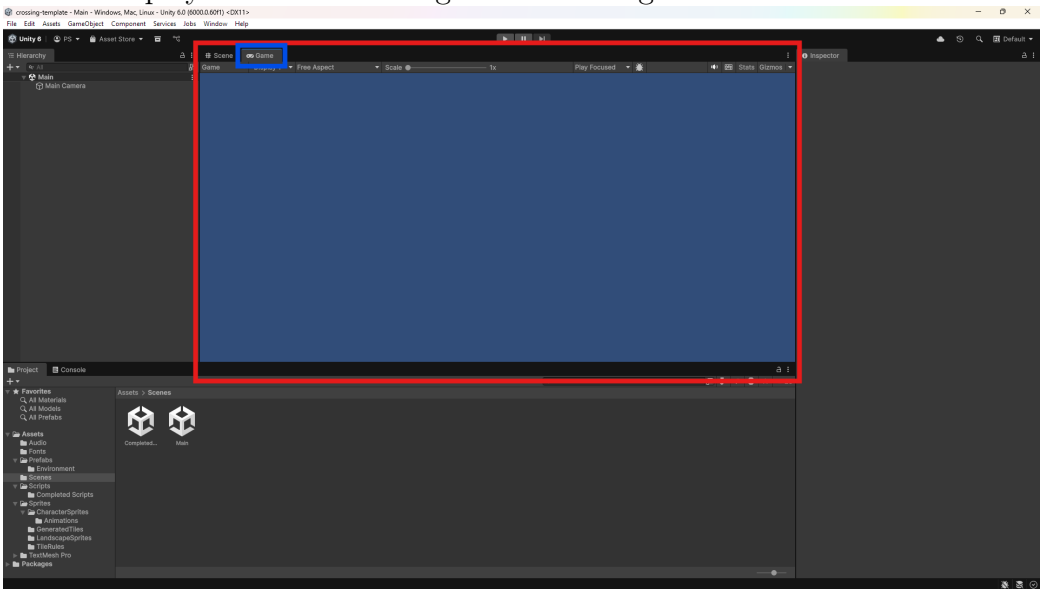
Scene View

Where you build and arrange your game world.



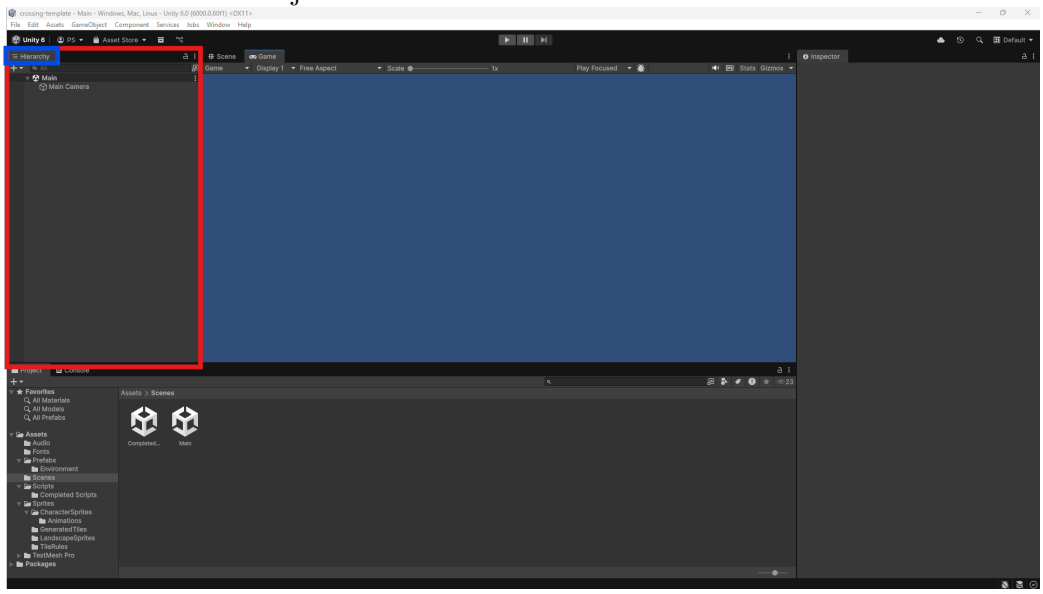
Game View

What the player sees when the game is running.



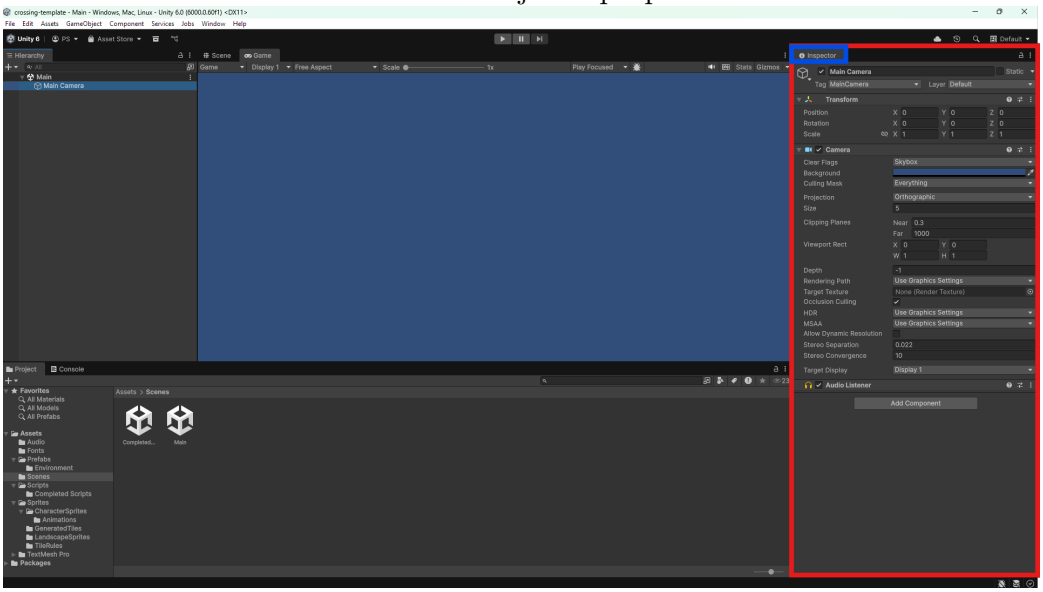
Hierarchy

A list of all GameObjects in the current scene.



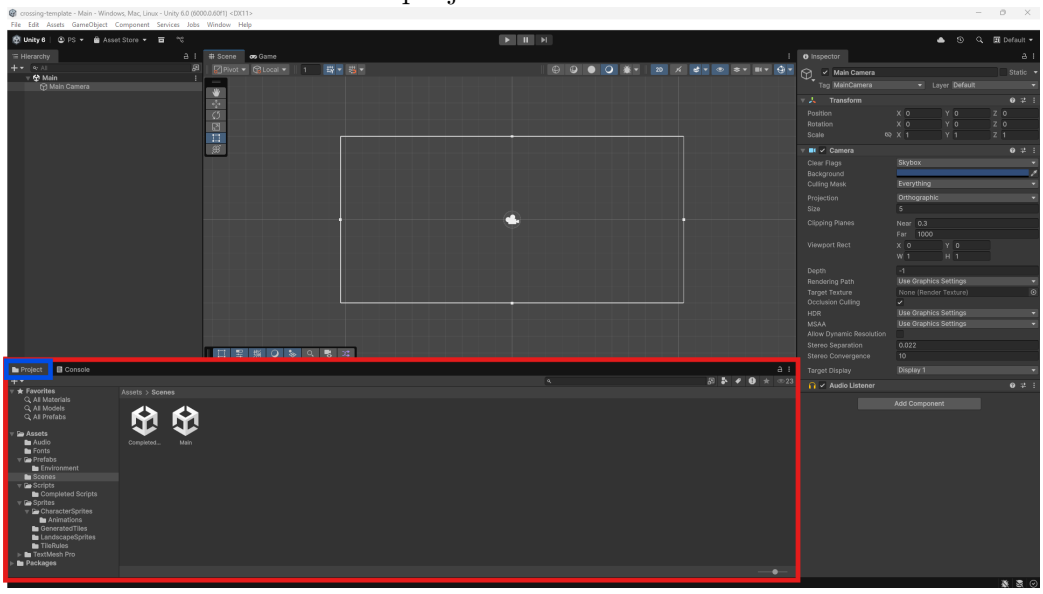
Inspector

Used to view and edit the selected object's properties.



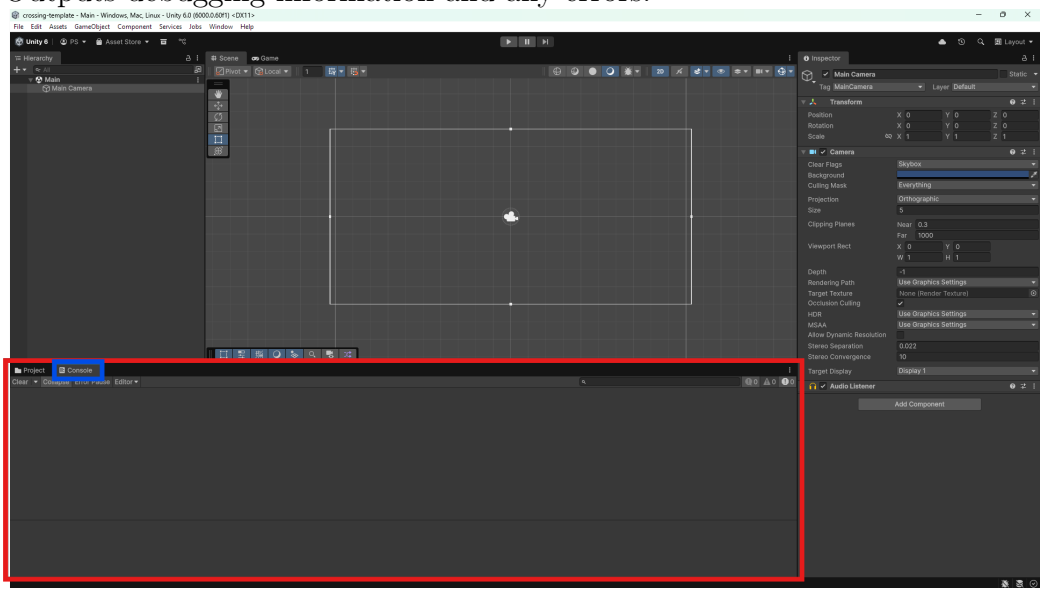
Project / Assets Window

Contains all files used in the project.



Console

Outputs debugging information and any errors.



2.2 Play Mode

The **Play** button allows you to test your game at any time. You will interact with the **Scene View**, **Game View**, and **Inspector** while testing.

- When Play Mode is active, the game runs as it would for a player.
- The Pause button pauses Play Mode without exiting.
- The Stop button exits Play Mode.

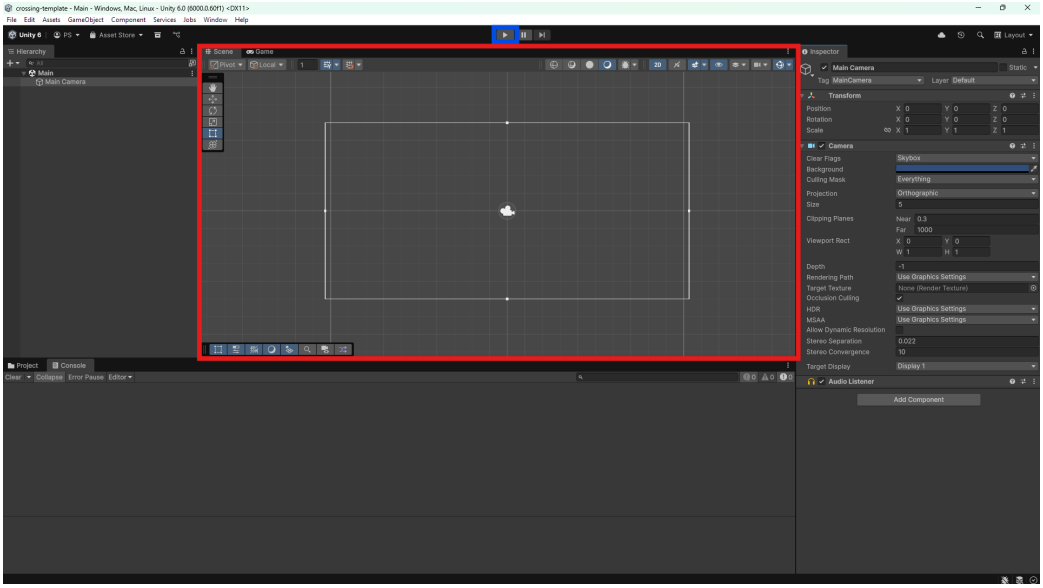
Warning

UNITY EDITOR CHANGES ARE DISCARDED AFTER EXITING PLAY MODE.

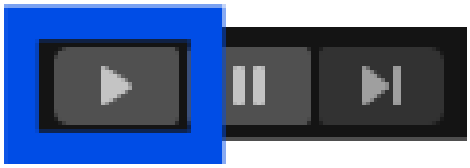
Make sure Play Mode is off or **you can lose hours of work!**

Note

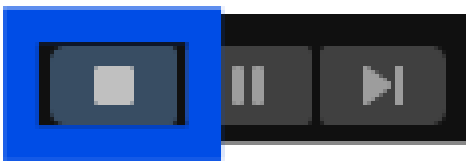
Play Mode doesn't discard changes you've made to your C# scripts or other code. But still good practice to turn off Play Mode!



(a) Unity Play button in the main toolbar



(b) Start Play Mode



(c) Exit Play Mode

3 Place Environment

3.1 Add Environment Pieces

- Make sure the "**Scene View**" tab is selected.
- Open the "Main" scene in "/Assets/Scenes".
- Right-click the **Hierarchy** and create: "2D Object → Tilemap → Rectangular".
- Delete the "Tilemap" object (don't delete the Grid!).
- In "Assets/Prefabs/Environment" hold "Shift" and left-click to select all objects, then drag them onto the "Grid" object in the **Hierarchy**.

3.2 Fix Camera Depth

- Select "Main Camera" in the **Hierarchy**.
- In the **Inspector**, set the camera's Z position to "-1".

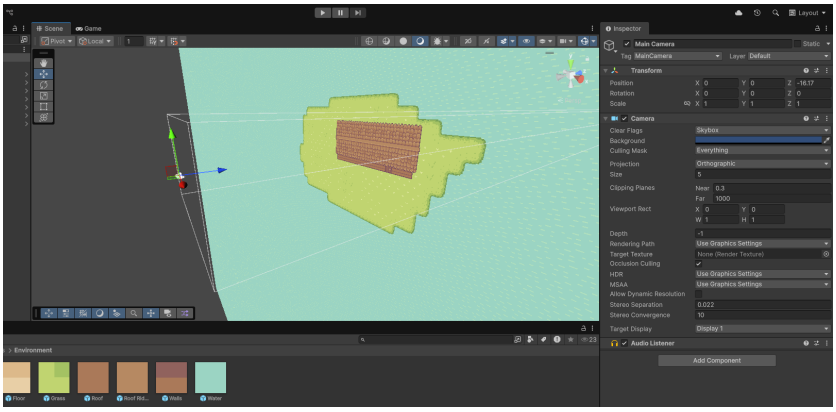


Figure 3: Even 2D is 3D in Unity!

4 Create Player

4.1 Add the Player Object

- Create a "2D Object → Sprites → Capsule" in the **Hierarchy**.
- Select the "Capsule" object and rename it to "Player" in the **Inspector**.
- In the **Inspector**, under "Sprite Renderer", click the circle next to "Sprite" and select "birb_0".
- Set "Order in Layer" to "1".

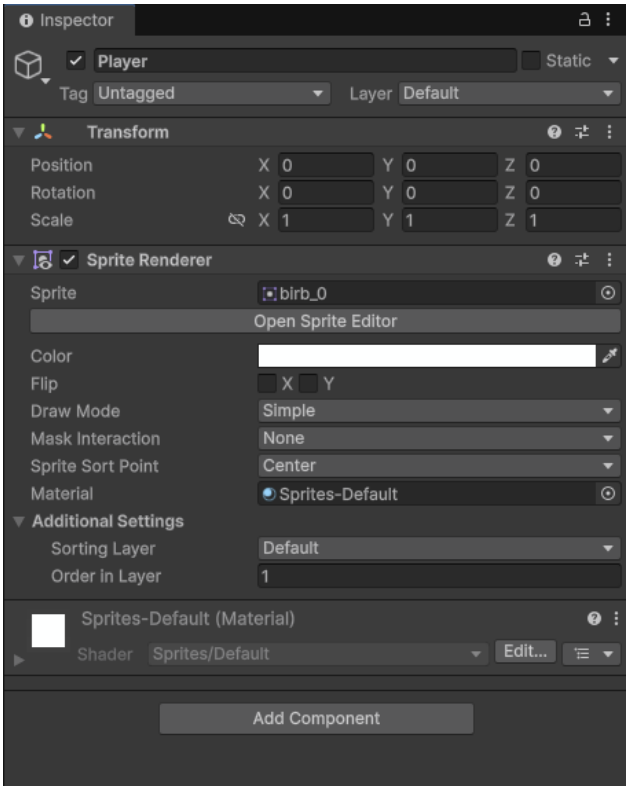


Figure 4: Player Inspector Window

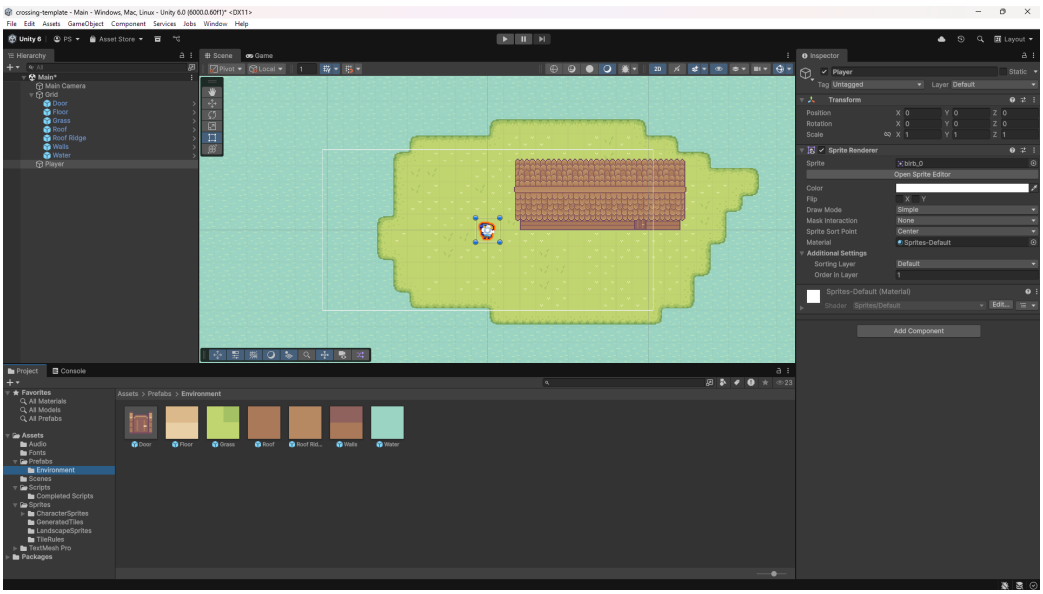


Figure 5: Current project progress!

5 Player Movement

5.1 Add Animator Component

- Select the Player in the **Hierarchy**
- In the **Inspector** click **Add Component**
- Search for **Animator** and then click on it to add it to the Player

5.2 Add Rigidbody Component

- With the player still selected, in the **Inspector** click **Add Component**
- Search for **Rigidbody 2D** and then click on it to add it to the Player
- Under Rigidbody 2D in the **Inspector** set **Gravity Scale** to 0 and find the Constraints dropdown and tick **Freeze Rotation - Z**

Warning

If you forget to lock the Z-Axis the player will bounce into 3D space and break the game!

5.3 Attach Player Movement Script

- Go to **"/Assets/Scripts"** and drag MoveCharacter.cs onto Player in the **Hierarchy**

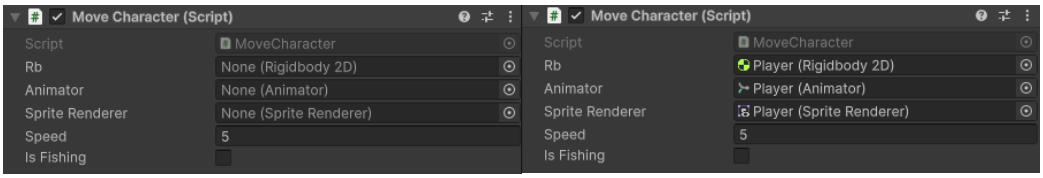
Note

For this workshop the scripts will be premade for you!
For future projects, to create a script right click in **"Assets/Scripts"** and find **"Create - Scripting - MonoBehaviour Script."**

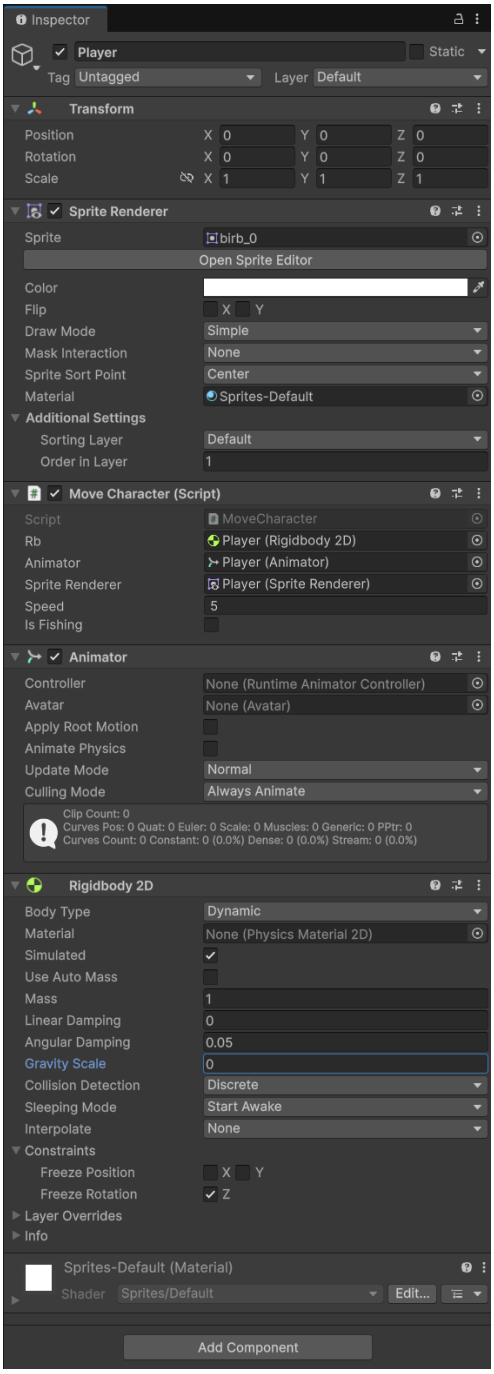
5.4 Assign Movement Script Variables

If a script does not have variables assigned "in-code" you need to fill those values in the Unity editor. You do that by dragging the correct references to the **Inspector**.

- Make sure Player is selected in the **Hierarchy**
- In the **Inspector** find **Move Character (Script)**
- Where you see **"None"** in the **Inspector**, drag the Player from the **Hierarchy** and place it 3 times in each **"None"** slot.



(a) Before Assignment *(b) After Assignment*



(a) Current Player inspector

5.5 Opening MoveCharacter.cs

- In the **Project / Assets Window** open "Assets/Scripts/MoveCharacter.cs"
- You should now see the MoveCharacter script in your IDE

```
using UnityEngine;

public class MoveCharacter : MonoBehaviour
{
    public Rigidbody2D rb;
    public Animator animator;
    public SpriteRenderer spriteRenderer;
    public float speed = 5f;

    public bool isFishing = false;

    private Vector2 moveDirection;

    void Start()
    {
        if (rb == null)
            rb = GetComponent<Rigidbody2D>();
    }

    void Update()
    {
        // TODO: Stop player from starting movement if
           fishing

        moveDirection = new Vector2(0, 0); // TODO:
           Make player move
        animator.SetFloat("Speed", 0);     // TODO:
           Tell animator when player moves

        if (moveDirection.x != 0)
        {
            SetFacingDirection(moveDirection.x);
        }
```

```
31      }
32
33      private void SetFacingDirection(float x)
34      {
35          float scaleX =
                Mathf.Abs(transform.localScale.x);
36
37          if (x < -0.01f)
38              scaleX = -scaleX;
39
40          transform.localScale = new Vector3(scaleX,
                transform.localScale.y,
                transform.localScale.z);
41      }
42
43      void FixedUpdate()
44      {
45          if (!isFishing)
46              rb.linearVelocity = moveDirection * speed;
47          else
48              rb.linearVelocity = Vector2.zero;
49      }
50  }
```

Listing 1: MoveCharacter.cs — Player movement logic

5.6 Unity Scripts

Most of the programming you do in Unity is with **MonoBehaviour** scripts. You don't need to know every detail yet. Generally, a MonoBehaviour is just a special kind of C# class that Unity provides, which comes with built-in functionality for gameplay, like responding to input, updating every frame, and interacting with GameObjects.

The important thing to know is that when you see this:

```
public class MoveCharacter : MonoBehaviour
```

It means your class inherits all the special behavior and lifecycle methods that Unity provides, such as `Start()`, `Update()`, and `FixedUpdate()`. This allows your script to automatically run code at the right times in your game.

Unity Script Lifecycle Methods

- **Start():** Runs once when the game begins. In `MoveCharacter.cs`, it's just double checking if Rigidbody2D actually exists when the game starts and grabs it if not.
- **Update():** Runs every frame. This is where we check player input, update movement direction, and tell the animator which animation to play.
- **FixedUpdate():** Runs at a fixed time interval, not every frame. This is usually where you want to do things like movement logic since then your player won't move faster if the game runs faster.

Think of it like this:

- `Start()` = "Set things up once at the beginning."
- `Update()` = "Do stuff that happens every frame, like checking input or animations."
- `FixedUpdate()` = "Move things that use physics at a steady, predictable pace."

To save time I have already filled most of the script, but there are a couple lines that are incomplete

```
1         void Update()
2     {
3         // TODO: Stop player from starting movement if
              fishing
4         moveDirection = new Vector2(0, 0); // TODO:
              Make player move
5         animator.SetFloat("Speed", 0); // TODO: Tell
              animator when player moves
6
7         if (moveDirection.x != 0)
8         {
9             SetFacingDirection(moveDirection.x);
10        }
11    }
```

Listing 2: MoveCharacter.cs — Update()

5.7 Finishing MoveCharacter.cs

So far, the player isn't actually moving yet. In `Update()`, we were setting the movement direction to `(0, 0)`, which means "don't move".

Now we replace that placeholder code with real input from the keyboard (we'll worry about stopping fishing movement later).

```
1    void Update()
2    {
3        // TODO: Stop player from starting movement if
             fishing
4
5        moveDirection = new Vector2(
6            Input.GetAxisRaw("Horizontal"),
7            Input.GetAxisRaw("Vertical")
8        ).normalized;
9
10       animator.SetFloat("Speed",
             moveDirection.sqrMagnitude);
11
12       if (moveDirection.x != 0)
13       {
14           SetFacingDirection(moveDirection.x);
15       }
16   }
```

Listing 3: MoveCharacter.cs — Update()

- `Input.GetAxisRaw("Horizontal")` reads left/right input (A/D or arrow keys).
- `Input.GetAxisRaw("Vertical")` reads up/down input (W/S or arrow keys).
- These values are combined into a `Vector2` called `moveDirection`.
- `normalized` keeps movement speed consistent in all directions.

This direction is not applied immediately. Instead, it is passed down to the physics step below.

```
1     void FixedUpdate()
2     {
3         // Removes any existing movement if fishing
4         if (!isFishing)
5             rb.linearVelocity = moveDirection * speed;
6         else
7             rb.linearVelocity = Vector2.zero;
8     }
```

Listing 4: MoveCharacter.cs — FixedUpdate()

`Update()` decides *which direction* the player should move, and `FixedUpdate()` applies that direction to the Rigidbody using physics.

5.8 Test the game!

Press play at the top of Unity and lets see what we have so far!

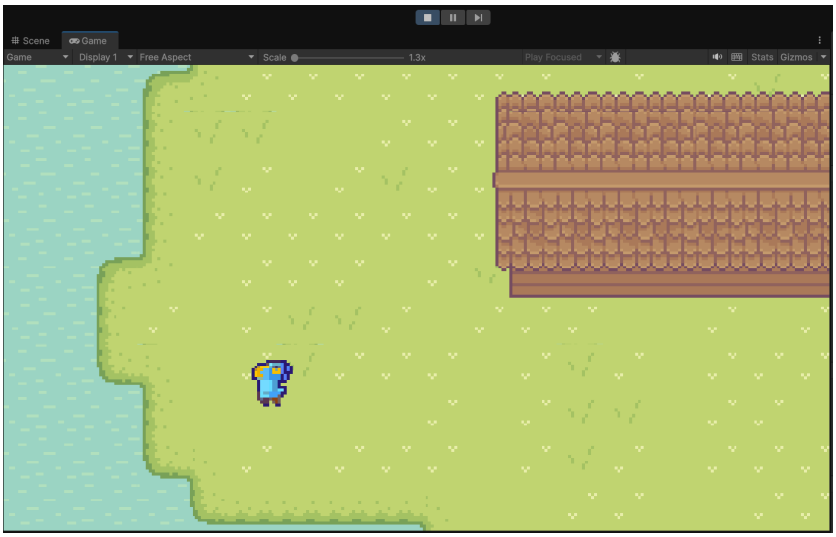


Figure 6: The player is moving but something is missing...

6 Finishing the Player

6.1 Add Camera Following Player

- In the **Hierarchy** take Main Camera and drag it on top of player
- Now the camera is following the player!

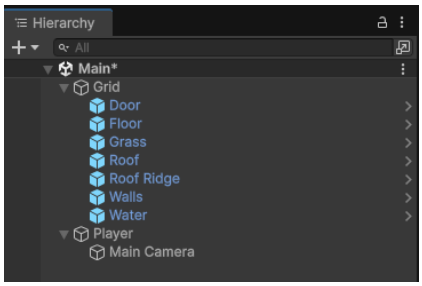


Figure 7: Hierarchy after making Main Camera a child of Player

6.2 Add Collision

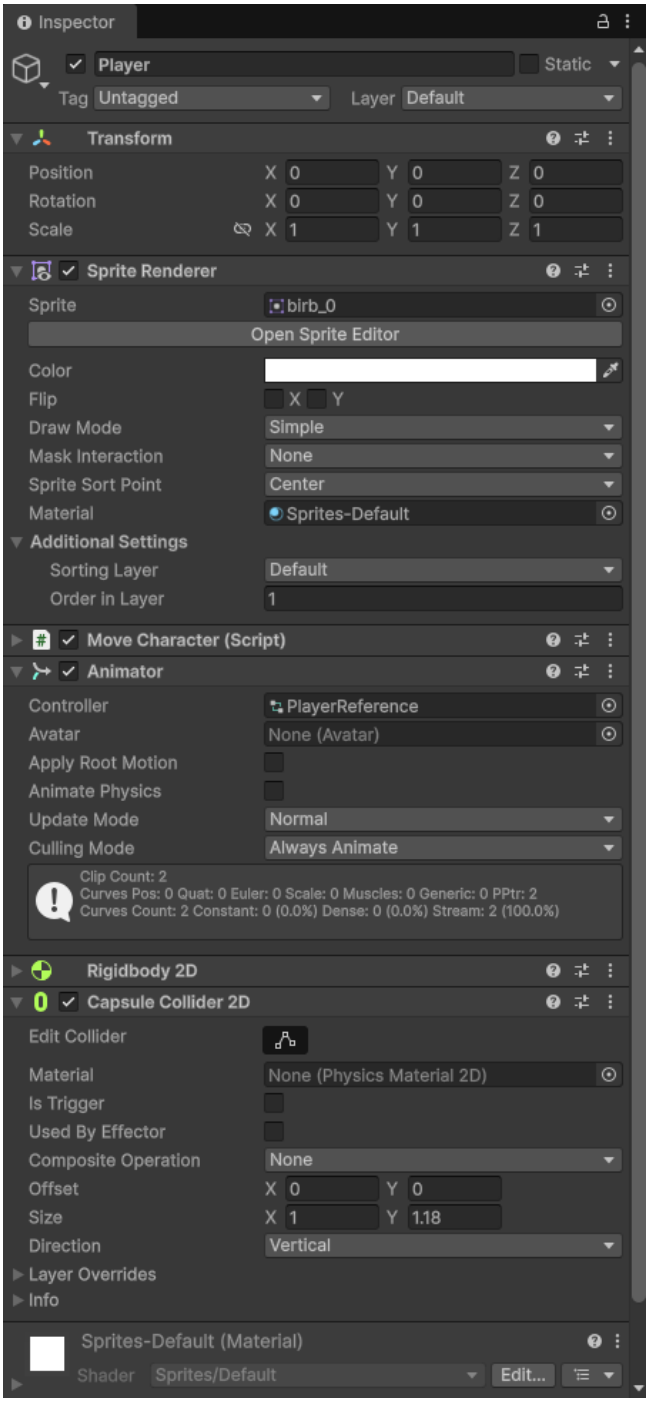
- Click **Player** in the **Hierarchy**
- In the **Inspector** click **Add Component** and then add a **Capsule Collider 2D**
- Adjust the **Size** attribute in the **Capsule Collider 2D** to **X = 1** and **Y = 1.2**

6.3 Add Walking Animations

- With the **Player** still selected, find the **Animator** component
- Under **Controller,** click the circle and select **PlayerReference** as the Animator Controller.

Note

We won't have time to create an animation controller for this workshop. However, if you're interested in setting one up yourself you can reference this video!: https://www.youtube.com/watch?v=vApG8aYD5aI



(a) Current Player Inspector

7 Fishing Mechanics (Rod, Fish, Script)

7.1 Add Fishing Rod and Fish Objects

- In "Assets/Prefabs" hold "Shift" and left-click to select **Fish** and **FishingRod**, then drag them onto the **Player**
- Select the **Fish** in the **Hierarchy** and then in the **Inspector** set **Y = 1.2**
- At the top of the **Inspector** untick both the **Fish** and the **FishingRod** to make them invisible.

7.2 Implement Basic Fishing Script

- In "Assets/Scripts" take **Fishing.cs** and place it on Player in the **Hierarchy**
- Double click **Fishing.cs** in "Assets/Scripts" to edit the script in an IDE

```
using UnityEngine;
using TMPro;
using System.Xml.Serialization;

public class Fishing : MonoBehaviour
{
    public GameObject fishingRod;
    public GameObject fishIcon;
    public TextMeshProUGUI scoreText;

    private bool nearWater = false;
    private bool fishingInProgress = false;
    // private int score = 0;
    private AudioSource[] audioSources;

    void Start()
    {
        audioSources = GetComponents<AudioSource>();
    }
```

```
20
21      // Update is called once per frame
22      void Update()
23      {
24          if (nearWater && !fishingInProgress &&
                Input.GetKeyDown(KeyCode.E))
25          {
26              // Start coroutine for fishing sequence
27              StartCoroutine(FishingSequence());
28          }
29      }
30
31      void OnTriggerEnter2D(Collider2D other)
32      {
33          if (other.CompareTag("GrassEdge"))
34          {
35              // TODO: Handle fishing enable
36          }
37      }
38
39      void OnTriggerExit2D(Collider2D other)
40      {
41          if (other.CompareTag("GrassEdge"))
42          {
43              // TODO: Handle fishing disable
44          }
45      }
46
47      // Coroutine (list of actions that run in order) to
            make the player fish
48      // Want to use this because you can wait without
            freezing the game
49      private System.Collections.IEnumerator
            FishingSequence()
50      {
51          fishingInProgress = true;
52
53          // Show fishing rod and play sound
54          if (fishingRod != null)
55          {
```

```
56              // TODO: Setup fishing start
57          }
58
59          // Stop player movement
60          MoveCharacter moveChar =
               GetComponent<MoveCharacter>();
61          // TODO: Stop the player from moving
62
63          yield return new WaitForSeconds(1.5f);
64
65          // TODO: Increase score and update text
66          if (scoreText != null)
67              scoreText.text = "Score Text";
68
69          // TODO: Hide fishing rod and icon
70
71          // TODO: Show fish icon
72
73          // TODO: Allow player movement again
74
75          // TODO: Hide fish after 1 second
76          yield return new WaitForSeconds(0.75f);
77
78          fishingInProgress = false;
79      }
80  }
```

Listing 5: Fishing.cs

We will mainly be editing **FishingSequence()**

7.3 FishingSequence()

FishingSequence() is a coroutine, which is basically list of things you execute in order without freezing the rest of the game to run. To that end, we're essentially going to show and hide the **Fish** and **FishingRod** objects we setup earlier based on what part of the coroutine is running.

```
1       private System.Collections.IEnumerator
           FishingSequence()
2       {
```

```
        fishingInProgress = true;

        // Show fishing rod and play sound
        if (fishingRod != null)
        {
            fishingRod.SetActive(true);
        }

        // Stop player movement
        MoveCharacter moveChar =
           GetComponent<MoveCharacter>();
        if (moveChar != null)
            moveChar.isFishing = true;

        yield return new WaitForSeconds(1.5f);

        // TODO: Increase score and update text
        if (scoreText != null)
            scoreText.text = "Score Text";

        if (fishingRod != null)
            fishingRod.SetActive(false);

        if (fishIcon != null)
        {
            fishIcon.SetActive(true);
        }

        if (fishingRod != null)
            fishingRod.SetActive(false);

        yield return new WaitForSeconds(0.75f);
        if (fishIcon != null)
            fishIcon.SetActive(false);

        fishingInProgress = false;
    }
```

Listing 6: Finished Coroutine

With these additions the player stops moving while fishing and also tog-

gles the Fishing Rod and Fishing Icon.

```
    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("GrassEdge"))
        {
            // TODO: Handle fishing enable
        }
    }

    void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("GrassEdge"))
        {
            // TODO: Handle fishing disable
        }
    }
```

Listing 7: Unfinished Tag Check

To finish checking for fishing spots we need to add a couple spots to toggle **nearWater**

```
    void OnTriggerEnter2D(Collider2D other)
    {
        if (other.CompareTag("GrassEdge"))
        {
            nearWater = true;
        }
    }

    void OnTriggerExit2D(Collider2D other)
    {
        if (other.CompareTag("GrassEdge"))
        {
            nearWater = false;
        }
    }
```

Listing 8: Finished Tag Check

7.4 Link Fishing Inspector References

- With the Player selected go to the **Inspector**
- Under the **Fishing** component in the **Inspector**, drag **Fish** and **FishingRod** from the **Hierarchy** and place them in the **Fishing** component

Last step before fishing works!!

7.5 Add Fishing Spots to the Environment

- In the **Hierarchy** right click and Create Empty
- With the empty object selected, go in the **Inspector** and rename it to **FishingSpot**
- Right below the name of the object adjust the Tag of the **FishingSpot** and set it to **GrassEdge**
- In the **Inspector**, **Add Component,** and add a **Box Collider 2D**
- In the **Box Collider 2D,** tick **Is Trigger** to on
- Adjust the size and position of the green **FishingSpot** square to change where your **Player** can fish!

Note

GrassEdge is a custom tag created for the workshop. Nothing special about it, but we're checking for **"GrassEdge"** in **Fishing.cs**. You can create your own tags by clicking **Add Tag...** in the Tag dropdown.

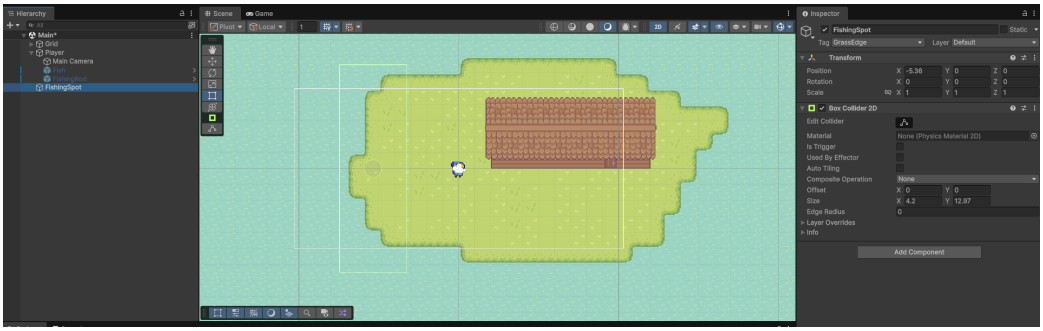


Figure 8: Scene with fishing spot added!

8 Finishing Touches (Sound, UI, Score)

8.1 Add Sound Effects

- Click the Player in the **Hierarchy** and **Add Component** in the **Inspector**
- Add an **Audio Source**, and repeat that to add a second **Audio Source**
- For the first **Audio Source** click the browse circle and select **Action**
- For the second **Audio Source** do the same and select **Collect 2**
- For both **Audio Sources** untick **"Play on Awake"**
- Now, open **Fishing.cs** for editing

```
1     private System.Collections.IEnumerator
          FishingSequence()
2     {
3         fishingInProgress = true;
4
5         // Show fishing rod and play sound
6         if (fishingRod != null)
7         {
8             fishingRod.SetActive(true);
9         }
10
```

```
        // Stop player movement
        MoveCharacter moveChar =
           GetComponent<MoveCharacter>();
        if (moveChar != null)
            moveChar.isFishing = true;

        yield return new WaitForSeconds(1.5f);

        // TODO: Increase score and update text
        if (scoreText != null)
            scoreText.text = "Score Text";

        if (fishingRod != null)
            fishingRod.SetActive(false);

        if (fishIcon != null)
        {
            fishIcon.SetActive(true);
        }

        if (fishingRod != null)
            fishingRod.SetActive(false);

        yield return new WaitForSeconds(0.75f);
        if (fishIcon != null)
            fishIcon.SetActive(false);

        fishingInProgress = false;
    }
```

Listing 9: Coroutine without sound

All we need to do here is use the **AudioSource** array in the script to play sounds at the correct time in the coroutine sequence.

```
private System.Collections.IEnumerator FishingSequence()
    {
        fishingInProgress = true;

        // Show fishing rod and icon
        if (fishingRod != null)
        {
```

```
8               audioSources[0].Play();
9               fishingRod.SetActive(true);
10          }
11
12          // Stop player movement
13          MoveCharacter moveChar =
                GetComponent<MoveCharacter>();
14          if (moveChar != null)
15              moveChar.isFishing = true;
16
17          yield return new WaitForSeconds(1.5f);
18
19          // TODO: Increase score and update text
20          if (scoreText != null)
21              scoreText.text = "Score Text";
22
23          // Hide fishing rod and icon
24          if (fishingRod != null)
25              fishingRod.SetActive(false);
26
27          // Show fish icon
28          if (fishIcon != null)
29          {
30              audioSources[1].Play();
31              fishIcon.SetActive(true);
32          }
33
34          // Allow player movement again
35          if (moveChar != null)
36              moveChar.isFishing = false;
37
38          // Hide fish after 1 second
39          yield return new WaitForSeconds(0.75f);
40          if (fishIcon != null)
41              fishIcon.SetActive(false);
42
43          fishingInProgress = false;
44      }
```

Listing 10: Coroutine with sound added on lines 8 & 31

Now, if we test out fishing in Play Mode our game will have sound!

8.2 Add UI Elements

- In the **Hierarchy** create a **"UI - Text - TextMeshPro"**
- If you'd like you can change the font and position of the text by adjusting the **Font Asset** and **PosX/PosY** in the **Inspector**.
- Select the **Player** in the **Hierarchy** and find the **Fishing** script in the **Inspector**
- Drag **Text (TMP)** from the **Hierarchy** to the **Score Text** field in the **Player's Fishing** script
- Last, we just need to open **Fishing.cs** and add the score logic.

Continuing from the script above, we just need to add a couple lines in the coroutine.

```
private System.Collections.IEnumerator FishingSequence()
    {
        fishingInProgress = true;

        // Show fishing rod and icon
        if (fishingRod != null)
        {
            audioSources[0].Play();
            fishingRod.SetActive(true);
        }

        // Stop player movement
        MoveCharacter moveChar =
           GetComponent<MoveCharacter>();
        if (moveChar != null)
            moveChar.isFishing = true;

        yield return new WaitForSeconds(1.5f);

        // Increase score and update text
        score++;
```

```
21          if (scoreText != null)
22              scoreText.text = "Score: " + score;
23
24          // Hide fishing rod and icon
25          if (fishingRod != null)
26              fishingRod.SetActive(false);
27
28          // Show fish icon
29          if (fishIcon != null)
30          {
31              audioSources[1].Play();
32              fishIcon.SetActive(true);
33          }
34
35          // Allow player movement again
36          if (moveChar != null)
37              moveChar.isFishing = false;
38
39          // Hide fish after 1 second
40          yield return new WaitForSeconds(0.75f);
41          if (fishIcon != null)
42              fishIcon.SetActive(false);
43
44          fishingInProgress = false;
45      }
```

Listing 11: Coroutine with score logic added on line 20

We also need to uncomment "score" at the top of the class.

```
1  public class Fishing : MonoBehaviour
2  {
3      public GameObject fishingRod;
4      public GameObject fishIcon;
5      public TextMeshProUGUI scoreText;
6
7      private bool nearWater = false;
8      private bool fishingInProgress = false;
9      // private int score = 0;
10     private AudioSource[] audioSources;
```

Listing 12: Commented score

to

```
public class Fishing : MonoBehaviour
{
    public GameObject fishingRod;
    public GameObject fishIcon;
    public TextMeshProUGUI scoreText;

    private bool nearWater = false;
    private bool fishingInProgress = false;
    private int score = 0;
    private AudioSource[] audioSources;
```

Listing 13: Uncommented score

Now you can enter **Play Mode** and everything from movement, to score, to sound should all work!

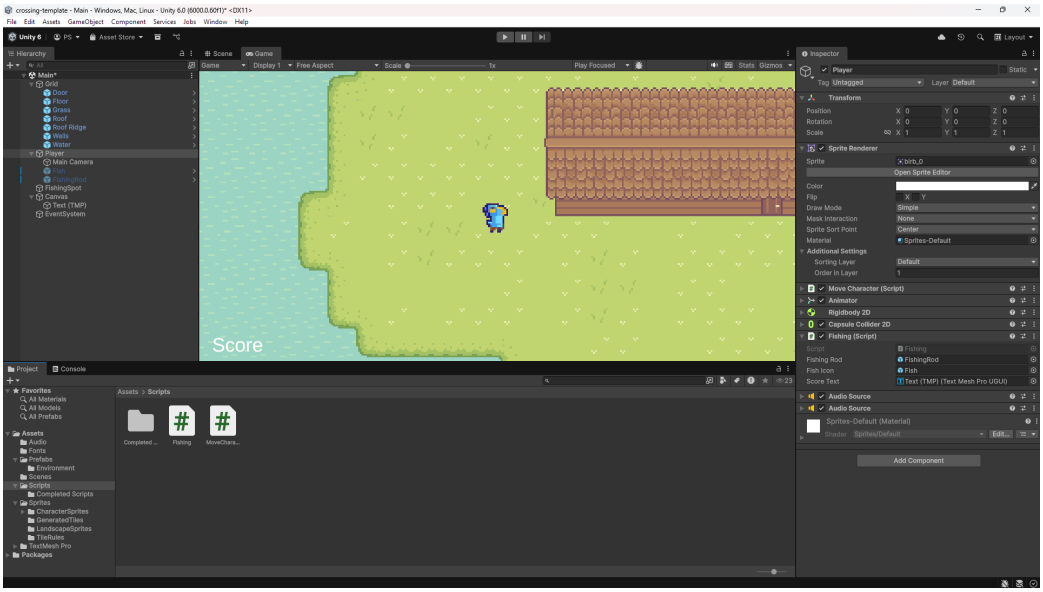


Figure 9: Final Completed Project

9 Where to Go Next

Congratulations!

You now have a working Unity game with player movement, animations, interaction, sound, and a basic gameplay loop.

Exporting Your Game

If you'd like to export this project and share it with others, Unity provides official documentation that walks through the process step by step:

- Unity Documentation — Building Your Game

Building Your World

We didn't spend much time on Unity's tileset system or how the environment used in the workshop was created. If you're interested in learning more about crafting levels with Tilemaps, this video is a great next step:

- Unity Tilemap Tutorial — Level Design Basics

Recommended Learning Resources

These are beginner-friendly, high-quality Unity resources:

- **Unity Learn (Official)**
  `https://learn.unity.com`
  Great structured tutorials straight from Unity!
- **Brackeys (YouTube)**
  `https://www.youtube.com/@Brackeys`
  The main channel for Unity basics!
- **Catlike Coding)**
  `https://catlikecoding.com/unity/tutorials/`
  Very thorough written tutorials!

- **Infallible Code**
  `https://www.youtube.com/@InfallibleCode`
  For when you're interested in taking the step up and making super clean Unity code!

Final Tip

Don't worry about making something "good" right away. The fastest way to learn Unity is to:

build small projects, break things, and experiment.

Thanks for dropping by the workshop and happy game developing!